

Companion for CallRadar & the Quality Checks

Turn 1,441 raw support-call recordings into a manager's dashboard — who called, what they wanted, how their mood moved, whether it was *really* resolved, which calls need attention today — and **the exact moment on the call that proves every judgment.**

Calls **1,441**Faithfulness **99.9%**Diarization **100%**Resolution-risk **195**API keys needed **0**

You get audio, not transcripts. Companion for CallRadar builds everything from the raw `.mp3`s exactly as they come off the phone system — stereo, 8 kHz, agent on the left channel and customer on the right. Every claim it makes is backed by a verified quote, and it measures its own quality the way the 2025 research literature does.

Results at a glance

Measured on the full 1,441-call run, served live at </api/evaluation>.

CALLS ANALYSED

1,441

the full dataset

CUSTOMERS · AGENTS

100 · 10

FAITHFULNESS

99.9%

evidence verified

DIARIZATION

100%

channel separation

COVERAGE

100%

well-formed outputs

RESOLUTION RISK

195sounded resolved,
wasn't

The brief, mapped to the implementation

Every requirement in the problem statement is implemented and verified on all 1,441 calls.

THE BRIEF REQUIRES	✓	WHERE
Speech to text	✓	<code>pipeline/transcribe.py</code> — faster-whisper
Work out who said what	✓	Channel-split (agent = L, customer = R) — correct by construction
Turn-by-turn with timings	✓	Interleaved turns + word-level timings
Per customer: name, history, recording + transcript	✓	Customers page → <code>/api/customers</code>
Per call: intent	✓	<code>analysis.intent</code>
Per call: mood + the point where it shifted	✓	<code>analysis.mood.shift.timestamp</code> + mood timeline
Per call: resolution	✓	<code>analysis.resolution.status</code>
Per call: summary (≤ 40 words)	✓	Validated ≤ 40 words on 100% of calls
Needs a manager's attention, ranked	✓	Needs Attention page → <code>/api/attention</code>
Which issues are trending	✓	Trends page → <code>/api/trends</code>
Per-agent volumes, handle times, outcomes	✓	Agents page → <code>/api/agents</code>
Every judgment cites the moment — timestamp + words	✓	Every judgment carries <code>evidence{timestamp, quote, speaker}</code>
Do not re-transcribe on every request	✓	Precomputed once → PostgreSQL; the API only reads
Git repo + README to run from scratch	✓	This repo; <code>docker compose run --rm pipeline</code>

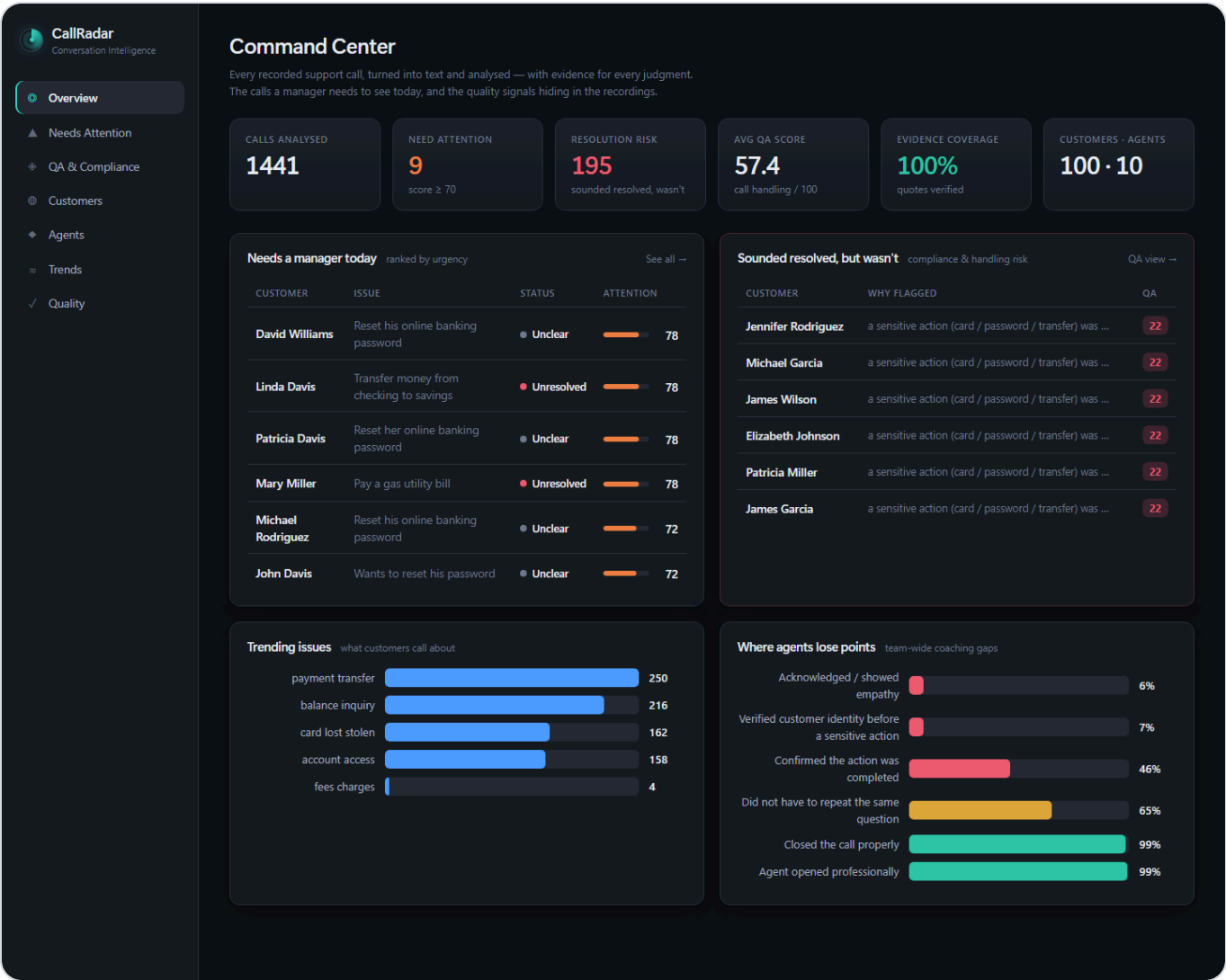
Why we stand out

Most entries produce a transcript, an LLM summary, and a table. Here's the difference:

A TYPICAL ENTRY	COMPANION FOR CALLRADAR
An ML diarizer <i>guesses</i> the speakers, and gets some wrong	Splits the stereo channels — attribution is correct by construction. Auto-corrects the ~37 reversed-channel recordings
The model <i>says</i> it's citing a moment; nobody checks	Every quote is verified against the transcript at its timestamp — 99.9% faithfulness measured
"It works" / an unverifiable "X% accurate"	We measure ourselves the way the 2025 research does — faithfulness, diarization, coverage
Summarises what was said	Flags the calls that sounded resolved but weren't — money moved with no identity check
Needs API keys / a specific cloud	Runs fully offline ; add Claude/Azure to upgrade

The dashboard & the API

Frontend — the manager's Command Center



Headline signals, the calls needing attention today, the "sounded resolved but wasn't" panel, and team-wide coaching gaps.

Per-call view — recording, transcript, evidence, mood, QA

CallRadar

Conversation Intelligence

Overview

Needs Attention

QA & Compliance

Customers

Agents

Trends

Quality

Back

Jennifer Rodriguez

Agent David · 0:48 · 3c4891900c3d4498 · engine: claude:claude-opus-4-8

Recording

AI summary

≤ 40 words

Jennifer called to replace a lost credit card. The agent confirmed which card but never verified her identity or explicitly confirmed a replacement was ordered, then abruptly ended the call without a timeframe.

lost credit card · card replacement · identity verification gap

Mood timeline

Neutral

Neutral

Judgments

every claim cites the moment that justifies it — click to hear it

INTENT — WHAT THEY WANTED

Replace a lost credit card - card lost stolen

00:17 · customer

✓ verified

"I lost my credit card. Can you send me a new one?"

MOOD — STEADY

Neutral

00:14 · customer

✓ verified

"Hi, my name is Jennifer Rodriguez"

RESOLUTION

Unclear

The agent confirmed the card type and closed the call, but never verified identity, confirmed a replacement was actually ordered, or provided a delivery timeframe. No explicit confirmation the card was reissued.

00:42 · agent

✓ verified

"Great. Thanks very much. Bye."

NEEDS-ATTENTION SCORE

65/100

Lost card report closed with no identity verification

No explicit confirmation a replacement card was ordered

Agent ended call abruptly without delivery timeframe

Possible security/process gap on a card-loss request

00:42 · agent

✓ verified

"Great. Thanks very much. Bye."

Evidence verification: 5/5 cited quotes matched the transcript at the stated timestamp.

QA & Compliance

how well the call was handled — every check cites the moment

22/100

RESOLUTION RISK — sounded resolved but wasn't

a sensitive action (card / password / transfer) was taken without verifying the customer's identity

the call closed politely but the requested action was never confirmed as completed

AGENT OPENED PROFESSIONALLY

PASS

00:01 · agent

✓ verified

"Hello. Hello, this is Harper Valley National Bank. My name is David. How can I help you today?"

VERIFIED CUSTOMER IDENTITY BEFORE A SENSITIVE ACTION

FAIL

A sensitive action (card/password/transfer) happened with no identity check. This is a compliance and fraud risk.

CONFIRMED THE ACTION WAS COMPLETED

FAIL

Agent never confirmed the request was actually done — the customer may leave unsure.

ACKNOWLEDGED / SHOWED EMPATHY

FAIL

Acknowledge the customer's situation before solving.

DID NOT HAVE TO REPEAT THE SAME QUESTION

FAIL

CLOSED THE CALL PROPERLY

PASS

00:28 · agent

✓ verified

"Yeah, thank you for letting me know. Let me see if I can help you replace your card. Which card would you like to replace? Your debit or credit card? Credit card? Perfect. And is there anything else I"

Agent asked the same thing more than once — listen actively / avoid rework.

Transcript

7 turns · agent = left channel, customer = right channel

Playable recording, turn-by-turn transcript, evidence-cited judgments (click a timestamp to hear it), mood timeline, and the QA / compliance breakdown.

Backend — the required API contract

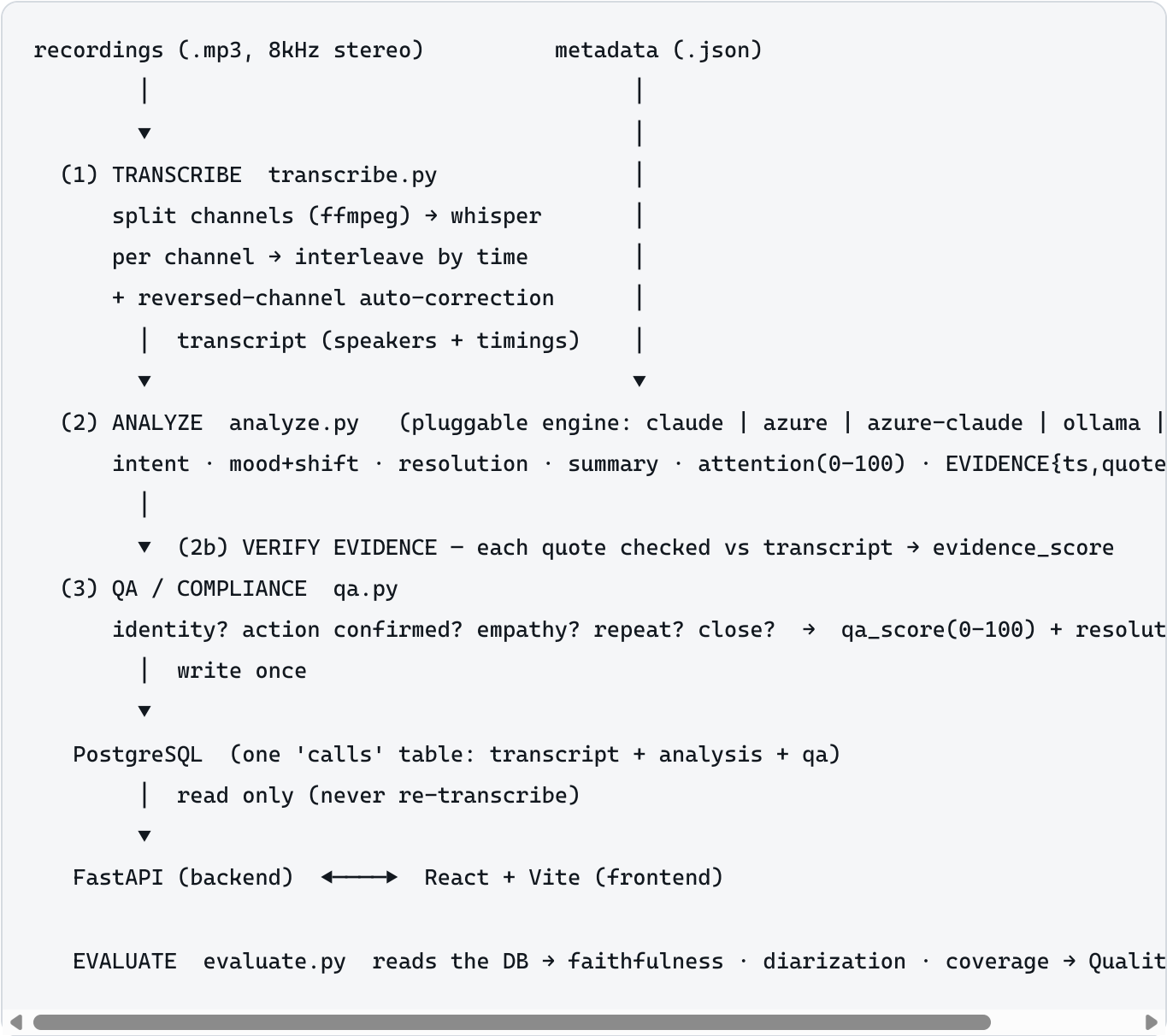
GET /api/calls/{sid} — the required per-call contract

```
{
  "sid": "3c4891900c3d4498",
  "customer_name": "Jennifer Rodriguez",
  "agent_name": "David",
  "started_at": "2020-05-30T17:57:47.377000",
  "duration_sec": 47.62,
  "intent_summary": "Replace a lost credit card",
  "intent_category": "card_lost_stolen",
  "mood_start": "neutral",
  "mood_end": "neutral",
  "mood_shifted": false,
  "resolution_status": "unclear",
  "summary": "Jennifer called to replace a lost credit card. The agent confirmed which card but never verified her identity or explicitly confirmed a replacement was made.",
  "attention_score": 65,
  "evidence_score": 1.0,
  "topics": [
    "lost credit card",
    "card replacement",
    "identity verification gap"
  ],
  "analysis_engine": "claude:claude-opus-4-8",
  "qa_score": 22,
  "resolution_risk": true,
  "session_label": "Little Harper Valley 2",
  "ended_at": "2020-05-30T17:58:39.583000",
  "stt_engine": "faster-whisper:base.en",
  "transcript": {
    "turns": [
      {
        "speaker": "agent",
        "start": 1.14,
        "end": 10.77,
        "text": "Hello. Hello, this is Harper Valley National Bank. My name is David. How can I help you today?",
        "words": [
          {
            "start": 1.14,
            "end": 1.64,
            "text": "Hello."
          },
          {
            "start": 6.21,
            "end": 6.61,
            "text": "Hello,"
          },
          {
            "start": 6.93,
            "end": 7.07,
            "text": "this"
          },
          {
            "...": "..."
          }
        ]
      },
      {
        "speaker": "customer",
        "start": 14.74,
        "end": 16.4,
        "text": "Hi, my name is Jennifer Rodriguez.",
        "words": [
          {
            "start": 14.74,
            "end": 15.04,
            "text": "Hi,"
          },
          {
            "start": 15.22,
            "end": 15.34,
            "text": "my"
          },
          {
            "start": 15.34,
            "end": 15.54,
            "text": "name"
          }
        ]
      }
    ]
  }
}
```

GET /api/calls/{sid} returns the transcript with speakers + timings, intent, mood + shift, resolution, ≤40-word summary, 0–100 attention score, QA, and the evidence behind every judgment.

Architecture

Data flows one way at build time (audio → transcript → analysis → QA → Postgres) and one way at request time (Postgres → API → UI). The two are decoupled: the pipeline can be re-run or swapped to a different engine without touching the serving layer, and the API never pays transcription cost.

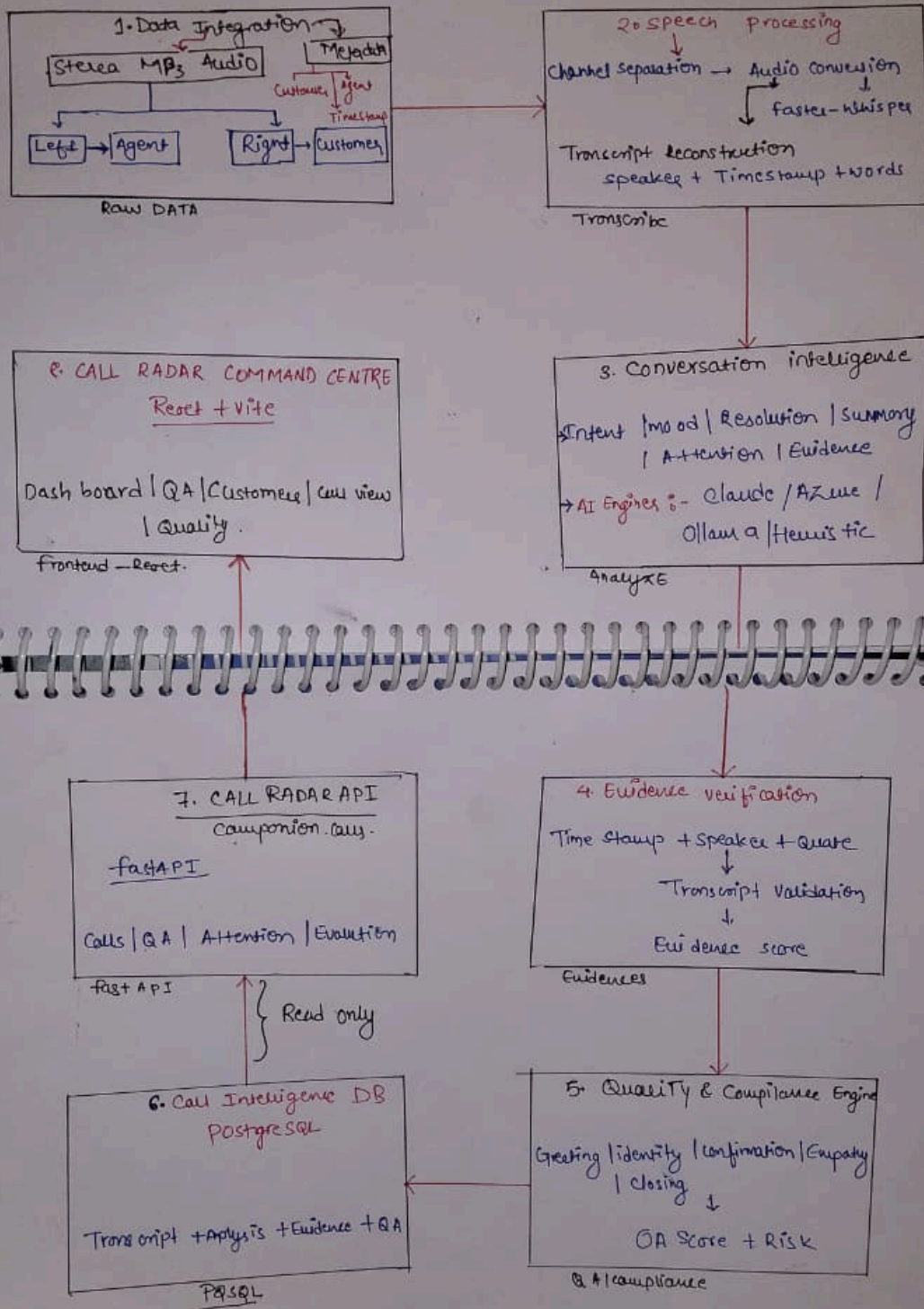


Hand-written design & formulation

The full architecture, the logic behind each stage, and every formula — worked out by hand.

Full Architecture :- implementation Along with formulations:-

3 QENTEC



Full architecture: raw audio → transcript → analysis → evidence → QA → DB → API → dashboard.

→ Formulation & Notations:-

Diarization - why channels, not guessing

let

'L' → agent

'R' - customer

we transcribe each separately, giving turn sets like 'T_L' and 'T_R'

so transcript the final one is like

$T = \text{sort}(T_L \cup T_R)$ order by turn start

→ separation quality:-

let me prove the channels really are separate speaker by the person Correlation

'p' between the two 'channels' energy envelopes "e_L", "e_R" (rooms window):-

so formulation:-

$$\rho = \frac{\sum (e_{-L_i} - \bar{e}_{-L})(e_{-R_i} - \bar{e}_{-R})}{\sqrt{[\sum (e_{-L_i} - \bar{e}_{-L})^2 \cdot \sum (e_{-R_i} - \bar{e}_{-R})^2]}}$$

$$\text{separation_quality} = 1 - \text{Max}(0, \rho)$$

Low (even negative) 'p' $\Rightarrow$ the two speakers don't overlap $\Rightarrow$ attribution is correct & "by construction"

measured on this dataset that given

$$\rho \approx -0.13$$

$$\text{separation_quality} = 1.00 \text{ (100\%)} \text{ really.}$$

$\rightarrow$ Evidence verification -

normalise(c) = lowercase(c) with only [a-z-0-9]

verified(c-j) = 1 if normalise(quite) $\leq$ normalise

(z-test) or $|Q_{HB}| / |Q| \geq 0.60$

otherwise

where Q = words (quote), B = words (z-test)

Pearson correlation ρ , separation quality, and evidence-verification logic.

per call evidence score.

$$\text{evidence score} = \frac{1}{n} \times \sum_j \text{verified}(e-j)$$

system wide faithfulness (over all calls c);

$$\text{faith} = \frac{\sum_c \sum_j \text{verified}(e-\{c,j\})}{\sum_c (\# \text{ judgments inc})}$$

$$= 0.999 \quad (99.9\%)$$

Attention Score (Need)

$$A \in [0, 100]$$

Per-call evidence score, system-wide faithfulness, and the needs-attention score.

QA / Compliance score $Q \in [0, 100]$

$$Q = \frac{\sum_k p_k \cdot w_k}{\sum_k w_k} \cdot 100$$

Over checks with $w_k > 0$

Number of quote (full run 1441 calls)

parameter	Score
calls	1441
customers	100
agents	10
faithfulness	99.9%
clauzation	100%
convergence	100%
resolution risk	195

QA / compliance score and the measured results on all 1,441 calls.

The pipeline — five stages

1 • Transcribe (channel-split diarization)

The recordings are stereo — agent left, customer right. Instead of guessing "who spoke" with an ML diarizer, we split the channels and transcribe each independently, then interleave by timestamp. Speaker attribution is correct **by construction**.

```
# c0=c0 = left(agent), c0=c1 = right(customer)
ffmpeg -i call.mp3 -af "pan=mono|c0=c0" -ar 16000 -ac 1 agent.wav
all_turns = transcribe(agent_wav, "agent")
all_turns += transcribe(cust_wav, "customer")
all_turns.sort(key=lambda t: t.start) # rebuild the real conversation
```

2 • Analyze (evidence-cited, pluggable engine)

A single tool whose input schema *is* the analysis schema forces the model to return every judgment with an `{timestamp, quote, speaker}` evidence object. Runs with no key (heuristic), or upgrades to Claude / Azure OpenAI.

2b • Verify evidence

Each cited quote is checked against the transcript turn at that timestamp (fuzzy, $\geq 60\%$ word overlap). Unverified citations are flagged. This is how we defend against the "wrong evidence scores negative" rule.

3 • QA / compliance

A bank-grade rubric scored from the transcript: identity verified before a sensitive action? action confirmed? empathy? repeated question? professional open/close? → a 0–100 QA score and a **resolution-risk** flag for calls that closed politely but were never actually completed.

4 • Store & 5 • Serve

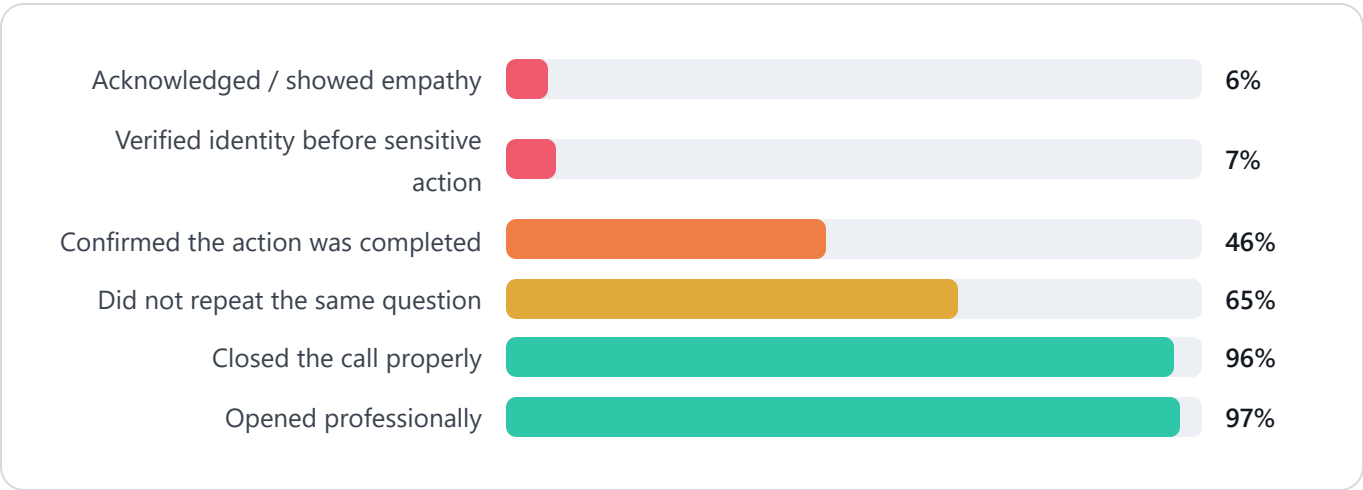
Everything is written once into a PostgreSQL `calls` table. FastAPI only reads — it never re-transcribes on a request.

Measurement criteria & results

We do **not** claim a single brittle "accuracy %". The conversation-intelligence and LLM-evaluation literature (WER for speech; FaithBench / FaithJudge / LLM-as-a-judge, 2025) measures **faithfulness** — is a claim grounded in the source? — and warns that exact-match scoring punishes good-but-reworded answers. So we measure on the axes that matter:

CRITERION	WHAT IT MEANS	HOW IT'S COMPUTED	RESULT
Faithfulness	Is every judgment backed by a quote that really appears at the cited time?	<code>verified / total judgments</code> ($\geq$ 60% word overlap)	99.9%
Diarization separation	Are the two speakers genuinely separate?	<code>1 - max(0, ρ)</code> , ρ = cross-channel energy correlation	100%
Coverage	Is every output well-formed?	summary $\leq$ 40 words, valid vocab, attention $\in [0,100]$, shift ts present, transcript present	100%
Avg QA score	How well were calls handled?	weighted compliance checks $\rightarrow$ 0–100	57.4
Resolution risk	"Sounded resolved but wasn't"	compliance / confirmation rule	195

Team-wide coaching gaps (from the QA layer)



Formulas & notation

Diarization — separation quality

Let L = agent channel, R = customer channel; transcribe each into turn sets T_L , T_R . The final transcript is $T = \text{sort}(T_L \cup T_R)$ by `turn.start`. To prove the channels are genuinely separate speakers, take the Pearson correlation ρ between the two channels' energy envelopes e_L, e_R (100 ms windows):

$$\rho = \frac{\sum (e_{L_i} - \bar{e}_L)(e_{R_i} - \bar{e}_R)}{\sqrt{[\sum (e_{L_i} - \bar{e}_L)^2 \cdot \sum (e_{R_i} - \bar{e}_R)^2]}}$$

$$\text{separation_quality} = 1 - \max(0, \rho)$$

Measured on this dataset: $\rho \approx -0.13 \rightarrow \text{separation_quality} = 1.00$ (100%)

Low (even negative) $\rho \Rightarrow$ the two speakers don't overlap $\Rightarrow$ attribution is correct by construction.

Evidence verification & faithfulness

Each judgment j carries evidence $e_j = (\text{timestamp}, \text{quote}, \text{speaker})$.

Verification finds the transcript turn τ at that timestamp+speaker and accepts if the quote appears in the turn *or* shares $\geq 60\%$ of its significant words:

`normalise(s) = lowercase(s) keeping only [a-z0-9]`

`verified(e_j) = 1 if normalise(quote) $\subseteq$ normalise(τ .text)
or $|Q \cap B| / |Q| \geq 0.60$
0 otherwise ($Q = \text{words}(\text{quote})$, $B = \text{words}(\tau.\text{text})$)`

`per-call evidence score =  $(1/n) \cdot \sum_j \text{verified}(e_j)$`

$$\text{faithfulness} = \frac{\sum_c \sum_j \text{verified}(e_{\{c, j\}})}{\sum_c (\# \text{ judgments in } c)} = 0.999 \quad (99.9\%)$$

Needs-attention score $A \in [0, 100]$

```
A = 10                                     (base)
  + min(35, 12·neg)                        (customer negativity)
  + 30 escalated / 28 unresolved / 15 unclear
  + 20 if mood shifted to negative (-5 if recovered to positive)
  + 22 if HIGH_RISK topic / 10 if MED_RISK
  + 12 if agent repeated a question
  + 10 if dur ≥ 90s (5 if ≥ 70s)
A = clamp(A, 0, 100)

HIGH_RISK = {fraud_dispute, card_lost_stolen, complaint, account_access}
MED_RISK  = {transaction_issue, fees_charges, loan_mortgage, payment_transfer}
```

QA / compliance score $Q \in [0, 100]$

Each check `k` has a pass flag `p_k ∈ {0,1}` and a weight `w_k` (identity = 3, action = 2, greeting / empathy / no-repeat / close = 1 each):

```
Q = 
$$\frac{\sum_k p_k \cdot w_k}{\sum_k w_k} \times 100$$
      (over checks with  $w_k > 0$ )

resolution_risk =
  (sensitive_action ∧ ¬identity_verified)          # compliance
risk
  ∨ (sounded_resolved ∧ customer_asked ∧ ¬action_confirmed ∧ status ≠ resolved)
```

Example output (real, from the API)

A payment/card call that *looks* fine — polite close — but the system catches the compliance failure:


```
{
  "customer_name": "Jennifer Rodriguez",
  "intent_summary": "Replace a lost credit card",
  "mood_start": "neutral", "mood_end": "neutral",
  "resolution_status": "unclear",
  "summary": "Jennifer called to replace a lost credit card. The agent confirmed
              which card but never verified her identity or explicitly confirmed a
              replacement was ordered, then abruptly ended the call.",
  "attention_score": 65,
  "qa_score": 22,
  "resolution_risk": true,
  "evidence_score": 1.0,
  "resolution": {
    "status": "unclear",
    "evidence": { "timestamp": "00:42",
                  "quote": "Great. Thanks very much. Bye.",
                  "speaker": "agent", "verified": true }
  },
  "qa": {
    "resolution_risk_reasons": [
      "a sensitive action (card / password / transfer) was taken without
      verifying the customer's identity"
    ]
  }
}
```

Every field is backed by a verified quote — click any timestamp in the dashboard and it jumps the audio to that exact second.

Run it from scratch

Prerequisites: Docker + Docker Compose. Nothing else. Runs with **no API keys**.

```
# 1. Load the dataset into ./data (accepts the zip OR an extracted folder)
python scripts/load_data.py <callradar-data.zip>

# 2. Bring up Postgres, the API, and the dashboard
docker compose up -d --build

# 3. Turn the recordings into transcripts + analysis (THE key step)
docker compose run --rm pipeline

# 4. Add the QA / compliance layer + quality metrics
docker compose run --rm pipeline python -m pipeline.compute_qa
docker compose run --rm pipeline python -m pipeline.evaluate

# 5. Open the dashboard → http://localhost:3000 (API → http://localhost:8000)
```

Premium analysis (optional): set ANTHROPIC_API_KEY or the AZURE_OPENAI_* / AZURE_CLAUDE_* variables and run `docker compose run --rm pipeline python -m pipeline.enrich --top 100` to upgrade the highest-attention calls to LLM-grade analysis. The engine is model-agnostic.

API reference

ENDPOINT	RETURNS
GET /api/calls/{sid}	Full per-call: transcript (speakers + timings), intent, mood + shift, resolution, ≤40-word summary, attention score, QA, evidence
GET /api/customers	Every customer with call counts and peak attention
GET /api/customers/{name}	A customer's full call history
GET /api/attention	Ranked "needs a manager's attention today"
GET /api/qa	QA-risk-ranked calls ("sounded resolved but wasn't")
GET /api/qa/stats	Team-wide QA metrics and coaching gaps
GET /api/evaluation	System quality: faithfulness, diarization, coverage
GET /api/trends	Trending issues across all calls
GET /api/agents	Per-agent volumes, handle times, and outcomes
GET /api/audio/{sid}.mp3	The playable recording
GET /api/stats	Dashboard headline numbers

Live links & repository

RESOURCE	LINK
GitHub repository	github.com/SAHILKOSHRIYA/Companion_Radar_call_01
Dashboard (local)	http://localhost:3000
API (local)	http://localhost:8000
Architecture doc	architecture.html · ARCHITECTURE.md
Formulas & notation	formulas.html · HANDWRITTEN_REFERENCE.md
Demo walkthrough	demo.html · DEMO.md

Companion for CallRadar & the Quality Checks — conversation intelligence over 1,441 consumer-bank support calls.

Stack: faster-whisper · FastAPI · PostgreSQL · React/Vite · Docker Compose. Runs from scratch with no API keys.